Weekly Project – Prompt Engineering

Zero-shot • One-shot • Few-shot • CoT • Manual CoT • ToT

1. Project Overview

Prompt Comparison Lab is an interactive Streamlit application that allows students to submit the same task to an LLM using different prompting techniques and compare the generated responses. The purpose is to understand how prompt design affects clarity, relevance, structure, consistency, and usefulness of an LLM response.

2. Objective

- Understand the basic concept of prompt engineering.
- Implement and compare zero-shot, one-shot, and few-shot prompting.
- Experiment with structured reasoning prompts such as CoT and Manual CoT.
- Understand the idea of Tree of Thoughts (ToT) through multiple candidate approaches.
- Build a simple Streamlit application connected to an LLM API.
- Evaluate and document the effect of different prompts on output quality.

3. Learning Outcomes

- Students can distinguish between different prompting strategies.
- Students can design prompts for explanation, classification, SQL generation, and problem solving.
- Students can compare multiple LLM outputs for the same task.
- Students can explain why examples, roles, constraints, and explicit structures can change model output.
- Students can build a basic LLM-powered application in Python.

4. Technologies Used

Technology	**Purpose**
Python	Application development
Streamlit	Web interface
OpenAI API	LLM response generation
Prompt templates	Different prompting techniques
Git/GitHub	Version control and submission

5. Prompting Techniques

Technique	Meaning	Example / Use
Zero-shot	Give the task without an example.	Explain artificial intelligence.
One-shot	Give one example before the new task.	Show one question-answer example, then ask a new question.
Few-shot	Give two or more examples before the new task.	Provide several examples to demonstrate the desired pattern.
CoT	Ask the model to solve carefully and provide a concise explanation of key steps rather than exposing hidden reasoning.	Useful for structured problem solving.
Manual CoT	The student explicitly defines Step 1, Step 2, Step 3, etc.	Makes the reasoning structure visible and controllable.
ToT	Ask the model to consider multiple candidate approaches, evaluate them, and select one.	Useful when several solution paths are possible.

6. Application Workflow

The application follows this workflow:

User Task → Select Techniques → Build Prompt → Send to LLM → Generate Response → Compare → Evaluate → Conclusion

7. Suggested Experiment Tasks

- Explain artificial intelligence to a first-year student.
- Classify a product review as Positive, Negative, or Neutral.
- Generate an SQL query to find the top 5 products by sales.
- Solve a percentage/discount/tax problem.
- Summarize a short paragraph.
- Extract name, email, phone number, and other fields from text.

8. Project Folder Structure

The project structure is:

```
Prompt_Comparison_Lab/
│
├── app.py
├── prompt_templates.py
├── llm.py
├── requirements.txt
├── .env.example
├── README.md
│
├── screenshots/
│   ├── zero_shot.png
│   ├── few_shot.png
│   └── comparison.png
```

10. Installation

- Install Python 3.10 or above.
- Open the project folder in VS Code.
- Open Terminal in the project folder.
- Run: pip install -r requirements.txt
- Set the OPENAI_API_KEY environment variable.
- Run: streamlit run app.py
- Open the local Streamlit URL shown in the terminal.

10. How the Code Works

- app.py creates the Streamlit user interface.
- prompt_templates.py creates a different prompt for each technique.
- llm.py sends the selected prompt to the LLM API.
- The app displays each response in a separate comparison column.
- Students rate clarity, relevance, and format from 1 to 5.
- Students write a conclusion about which prompting technique performed best.